4회차 과제 — 전처리와 저장 이커머스 클릭스트림 추천 파이프라인

메타코드 데이터 엔지니어 7기 · 이유석 · 입력: Kafka user-events (3회차 producer 가 쌓은 RetailRocket 이벤트 23,000건) · 코드: preprocess.py, schema.sql

1. 처리 흐름 — 입력 → 변환 → 저장

Kafka user-events ——▶ Spark Structured Streaming (micro-batch: 쌓인 것 전부 / --continuous 1분) ——▶ foreachBatch
① JSON 파싱(스키마 고정) → ② 검증: 필수 필드 · event_type 허용값 · 시각 파싱 · transaction 인데 id 없음 ——실패——▶ user-events-dlq (원문 + dlq_reason)
③ event_id 로 배치 내 중복 제거 → ④ Parquet append (data/parquet/events/dt=날짜/) → ⑤ PostgreSQL events_staging (JDBC)
→ ⑥ events 로 INSERT … ON CONFLICT (event_id) DO NOTHING → ⑦ 이번 배치가 건드린 1분 윈도우만 events 에서 다시 세서 item_stats_1m 에 upsert

실행 로그 `batch=0 kafka=23000 parsed_ok=23000 dlq=0 dedup=23000(-0) | pg inserted=23000 skipped_existing=0 stats_upsert=22194 | 7.5s` — 읽은 수 · 파싱 성공 · DLQ · 중복 제거 후 · DB 신규/기존 · 집계 행을 따로 센다.

2. 입력 · 출력 예시 — 처리 전 / 처리 후, 그리고 왜 그 처리가 필요한가

처리 전 (Kafka value, key="693516")

```
{"event_id":"56b41caa-05fc-40da-ae34-a41c18e0fb5e",
 "event_type":"addtocart",
 "event_time":"2015-05-03T03:00:04.384Z",
 "produced_at":"2026-10-26T02:34:17.081Z",
 "schema_version":1, "source":"replay-generator",
 "payload":{"visitor_id":693516,"item_id":297662,"transaction_id"
:null}}
```

처리 후 events 1행 (PostgreSQL)

```
event_id      56b41caa-05fc-40da-ae34-a41c18e0fb5e (uuid, PK)
event_type    addtocart
event_time    2015-05-03 03:00:04.384+00 (timestamptz)
visitor_id / item_id  693516 / 297662 (bigint)
transaction_id NULL  ·  schema_version 1  ·  ingested_at
2026-10-28 03:50:55+00
```

집계 후 item_stats_1m: window_start 2015-05-04 05:54 · item 347132 · view 6 · cart 0 · tx 0 · visitors 1

항목	내 데이터에서 실제로 한 처리	이유
자료형	문자열 ISO 시각 → timestamptz, payload 숫자 → bigint, schema_version → int. 못 바꾸면 DLQ	JSON 은 전부 문자열/숫자 구분이 느슨하다. 집계 · 인덱스 · 범위 조회는 정확한 타입이 있어야 한다
시각	집계 기준은 event_time(원본 발생 시각, UTC). produced_at(보낸 시각) · ingested_at(저장 시각)은 따로 둔다	늦게 도착한 이벤트도 원래 분(minute)에 들어가야 한다. 세 시각 차이가 곧 end-to-end 지연 (6회차 지표)
결측 · 오류	필수 필드 없음 · event_type 이 view/addtocart/transaction 아님 · transaction 인데 transaction_id 없음 → 버리지 않고 DLQ 에 이유와 함께	깨진 메시지 하나 때문에 파이프라인을 멈추거나, 조용히 버려서 유실하면 안 된다. 실제 테스트: JSON 아님 · event_type=purchase 2건 → dlq=2
중복	배치 안: event_id dropDuplicates. 배치 간 · 재실행: PK(event_id) ON CONFLICT DO NOTHING. 집계는 더하지 않고 다시 계산	event_id 가 행 내용 해시(uuid5) 라 같은 행은 항상 같은 ID. 재시도 중복 · 재실행 중복 · 원본 중복을 한 기준으로 거른다

3. 최종 스키마 — 고유 키: events.event_id = uuid5(timestamp | visitorid | event | itemid) · item_stats_1m 은 (window_start, item_id)

events (정제 이벤트, 1행 = 메시지 1건)	타입	의미	item_stats_1m (1분 × 아이템 집계)	타입	의미
event_id	uuid PK	행 내용 해시. 중복 판단 · 멱등 저장 기준	window_start	timestamptz PK	date_trunc('minute', event_time)
event_type	text, check	view / addtocart / transaction	item_id	bigint PK	상품
event_time	timestamptz	원본 발생 시각(UTC). 윈도우 집계 기준	view_cnt / addtocart_cnt / transaction_cnt	int	종류별 건수
produced_at	timestamptz	생성기가 보낸 시각. 지연 측정	unique_visitors	int	그 분에 그 상품을 건드린 유저 수
visitor_id / item_id	bigint	유저 · 상품 (인덱스: visitor+time, item+time)	updated_at	timestamptz	마지막 재계산 시각
transaction_id	bigint null	구매일 때만 주문 ID	Parquet (data/parquet/events)		events 와 같은 컬럼 + dt 파티션(날짜). 원본 보관용
schema_version / source	int / text	메시지 형식 버전 · 출처	DLQ (user-events-dlq)		raw_value · partition · offset · dlq_reason · rejected_by
ingested_at	timestamptz	DB 에 들어온 시각 (default now())			

4. 저장 포맷과 선택 이유 — 조회는 PostgreSQL, 재처리는 Parquet

저장소	실제 저장 결과	선택 이유 (조회 · 재처리 요구 기준)	대안 / 트레이드오프
PostgreSQL events, item_stats_1m	events 23,000행 (view 22,354 · cart 523 · tx 123) · item_stats_1m 22,194행, 합계가 events 와 일치	유저별 · 아이템별 조회와 JOIN 이 필요(추천 Top-N, 트렌딩). 인덱스 + PK upsert 로 같은 입력을 몇 번 넣어도 결과가 같다	파일만 쓰면 조회마다 전체 스캔. 대신 운영 컴포넌트가 하나 늘고 적재가 배치 단위(staging → upsert) 로 한 번 더 간다
Parquet data/parquet/events/dt=…	dt=2015-05-03 8파일, dt=2015-05-04 8파일, 23,000행. 컬럼형 · snappy 압축	Kafka 는 7일이면 지워진다. 집계 로직에 버그가 나면 원본이 있어야 다시 돌릴 수 있다. 날짜 파티션이라 특정 날만 재처리 가능	CSV/JSON 보다 작고 빠르지만 사람이 바로 못 읽는다. append 라 재처리 때 중복 파일이 생김 → 읽을 때 event_id 로 거르거나 compaction 필요
Kafka DLQ user-events-dlq	깨진 메시지 2건 (invalid_json, bad_event_type)	실패 원인 분석 후 고쳐서 재전송할 수 있게 원문을 그대로 보관. 30일 리텐션	DB 테이블로 둘 수도 있지만 Kafka 안에 두면 재전송 경로가 같다

재실행 검증 체크포인트 유지 후 재실행 → 새 offset 없음, 저장 결과 변화 없음 | --reset 으로 처음부터 → inserted 0 · skipped_existing 23,000, item_stats_1m 내용 md5 전후 동일(fde20a24…), events 23,000 유지 | Parquet 만 46,000행(append) — README 5-3 남은 문제